Service start-up report

Generated: 2026-10-09 12:44:37
Target: siem (local)
Operation: **start** · Recorded window: 59s · compose exit code: **0**

Resource usage recorded while the stack was brought up. Each dashed red vertical line marks a major event (a container starting, going healthy or unhealthy, or stopping), numbered in a band above the graph; events within 1.5s of each other share one number, and the event key under the host graph says what each number covers. The largest CPU peak is called out with the marker it lines up with. **Figure 1** shows how these services fit together – the per-container graphs below track each box in it.

Reading the probability metrics. A **95% confidence interval (CI)** is the range the true value would fall in 95 times out of 100 if the measurement were repeated: a wide interval means few or noisy samples, a narrow one means the figure is well pinned down. A **p-value** is the probability of seeing a difference at least this large if there were really no effect; below 0.05 it is conventionally called *significant*, i.e. unlikely to be chance. Rates use the Wilson interval, which stays within 0–100% even for small samples or 0% / 100% results. Where a mean over a time series (CPU, memory) has a CI, it uses an effective sample size that discounts correlated neighbouring samples, so it is not over-confident. **P(≥x)** is the share of a recording spent at or above x: the chance that a randomly picked moment was that busy. **SD** (standard deviation) is the typical distance of a value from the mean.

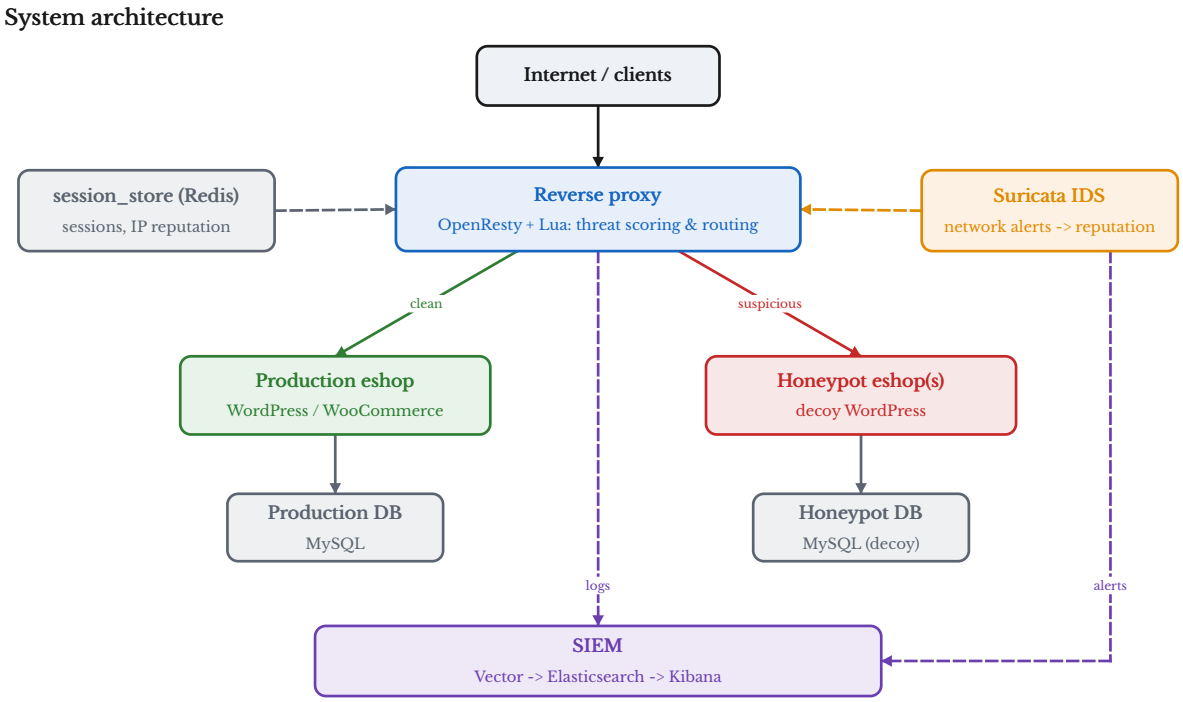
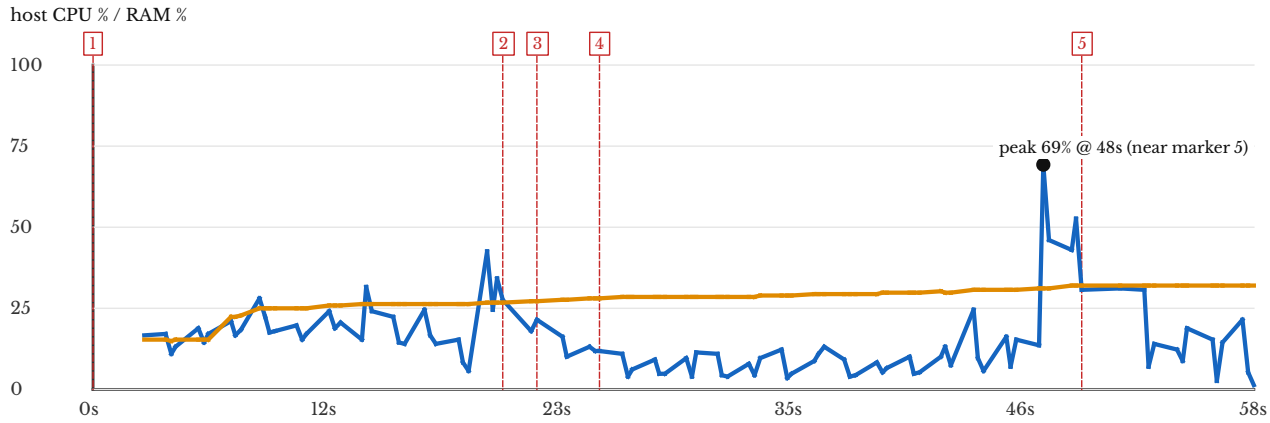


Figure 1. System architecture: the services whose CPU / memory the per-container charts below track as the stack is started.

1. Overall system (host)



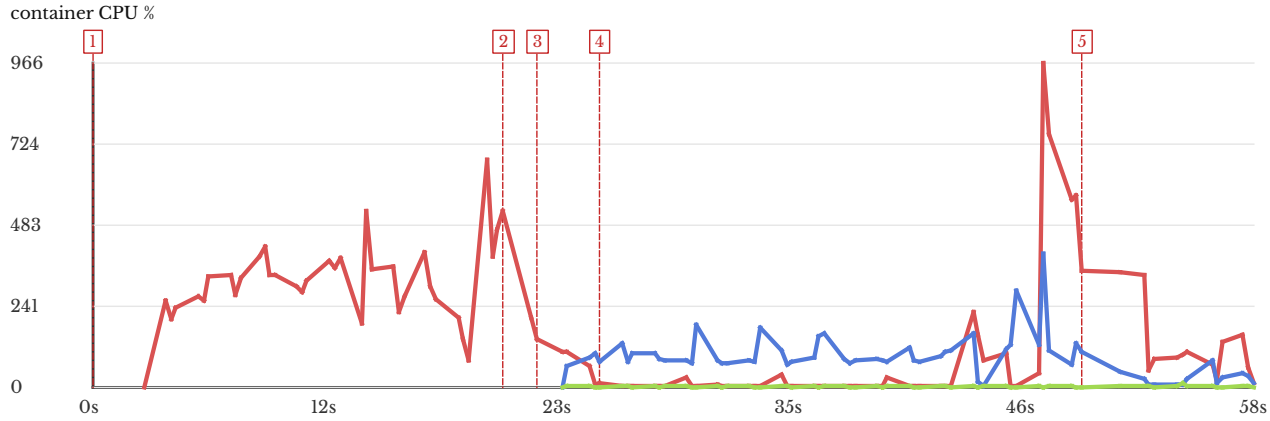
Event key (marker numbers on all graphs)

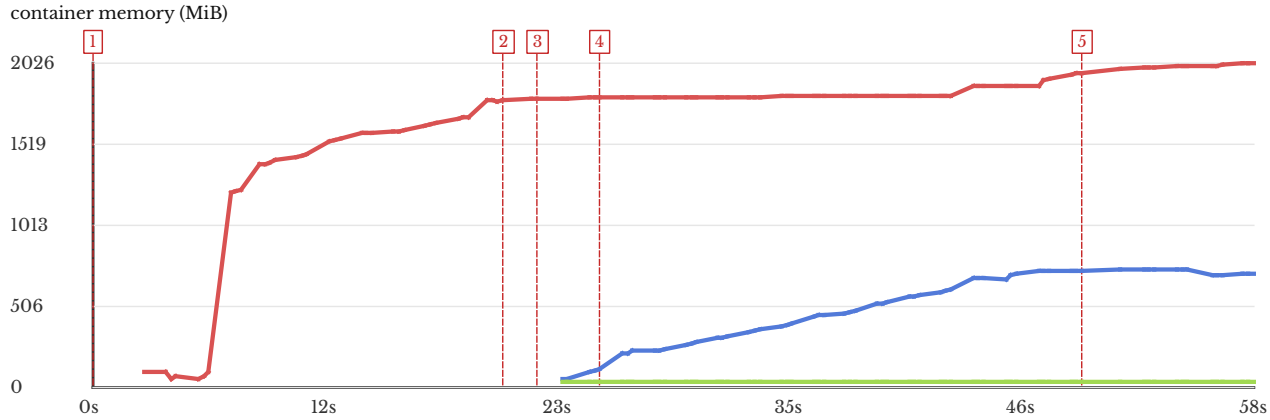
Marker	Elapsed	What happened
1	0s	started: init-password, kibana_dashboards_setup, es01, kibana, vector_inbound
2	20s	healthy: es01
3	22s	running, health pending: kibana, vector_inbound
4	25s	healthy: vector_inbound
5	49s	healthy: kibana; stopped: init-password

Probability metrics: host		
Host CPU, mean ± SD	**15.3% ± 11.2**	average over 101 samples; the SD is the typical swing around it
Host CPU, 95th percentile / peak	**34.2% / 69.1%**	95% of the time usage was at or below the first figure
Host CPU, how unusual the peak is	**4.8 SD above the mean**	more than ~3 SD is a rare spike rather than normal fluctuation
Host CPU, P(≥50%)	**2.0%**	chance a random moment of this recording was at least this busy
Host CPU, P(≥80%)	**0.0%**	chance a random moment of this recording was at least this busy
Host RAM used, mean ± SD	**27.4% ± 4.1**	average over 101 samples; the SD is the typical swing around it
Host RAM used, 95th percentile / peak	**31.8% / 31.9%**	95% of the time usage was at or below the first figure
Host RAM used, how unusual the peak is	**1.1 SD above the mean**	more than ~3 SD is a rare spike rather than normal fluctuation
Host RAM used, P(≥50%)	**0.0%**	chance a random moment of this recording was at least this busy
Host RAM used, P(≥80%)	**0.0%**	chance a random moment of this recording was at least this busy

2. Per-container

CPU and memory were read from the containers' cgroup files every 0.25 s.





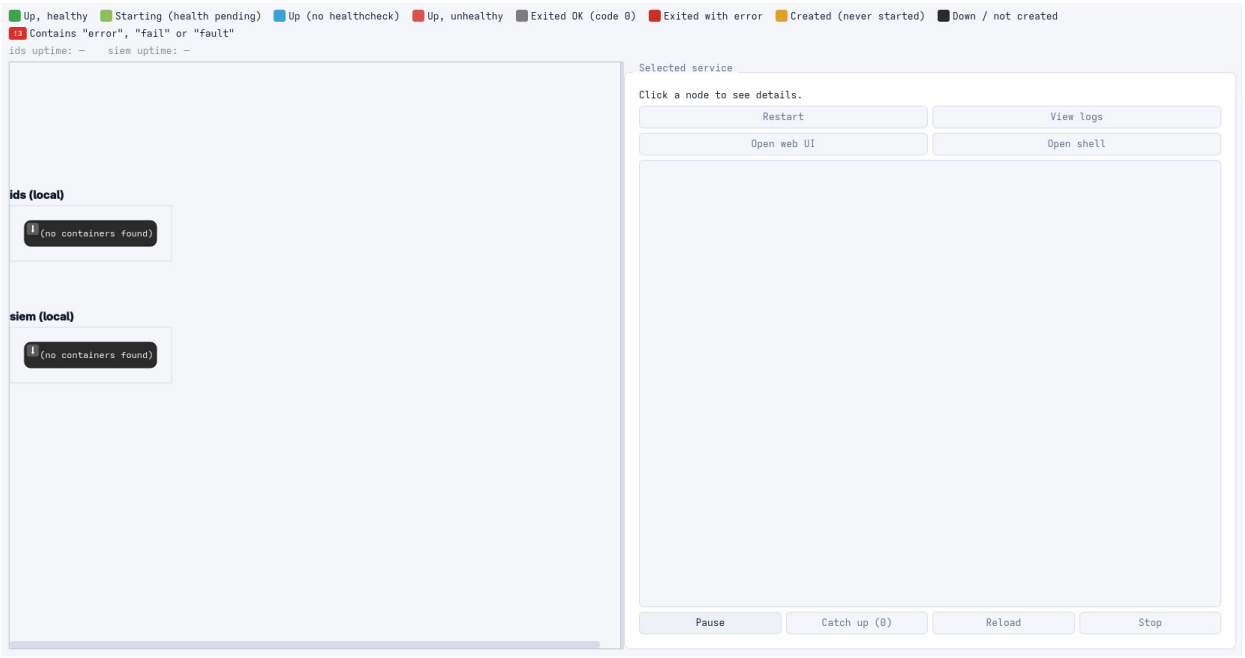
Legend: ■ es01 ■ kibana ■ vector_inbound

Probability metrics: per container					
Container	**CPU mean ± SD**	**CPU p95 / peak**	**P(CPU ≥ 50%)**	**Memory MiB, mean ± SD**	**Memory MiB, peak**
es01	172.1% ± 196.2	524.0% / 965.8%	59.4%	1651 ± 469	2026
kibana	85.5% ± 64.7	174.2% / 397.6%	76.2%	497 ± 217	731
vector_inbound	0.3% ± 1.6	0.1% / 12.7%	0.0%	33 ± 1	34

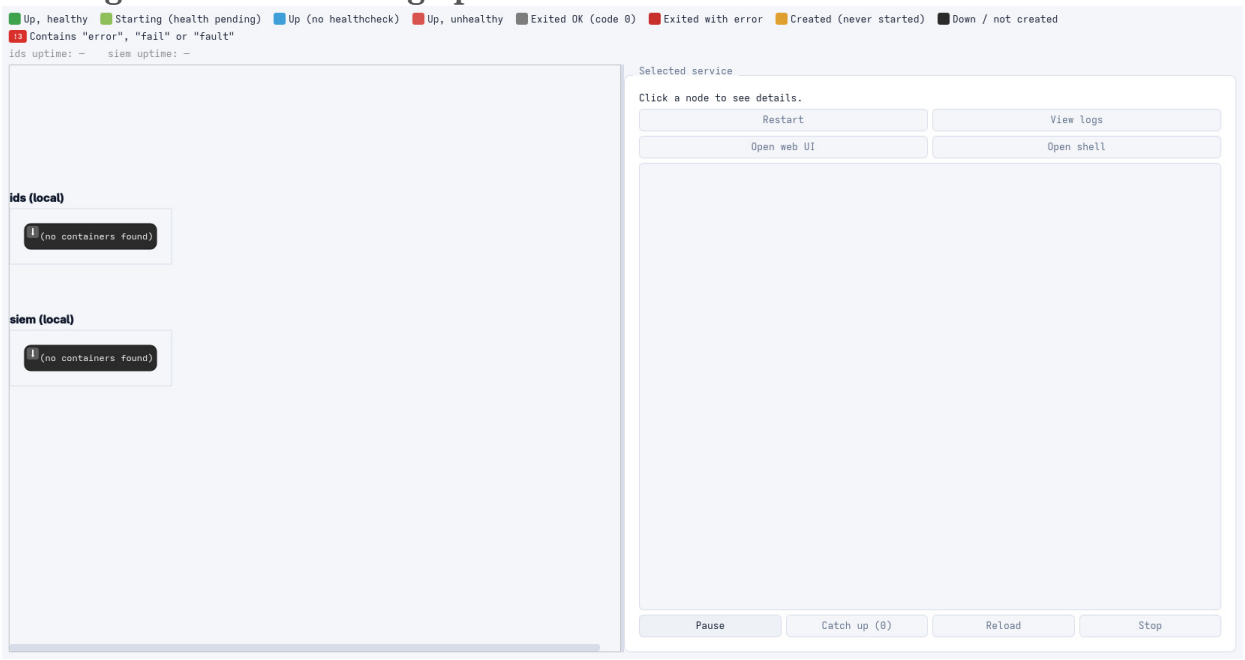
P(CPU ≥ 50%) is the share of the recording the container spent at half a core or more, i.e. the chance of catching it that busy at a random moment. A peak far above the p95 means a short burst (typically start-up work or a first healthcheck) rather than sustained load.

3. Global Visualization page as the stack changes state

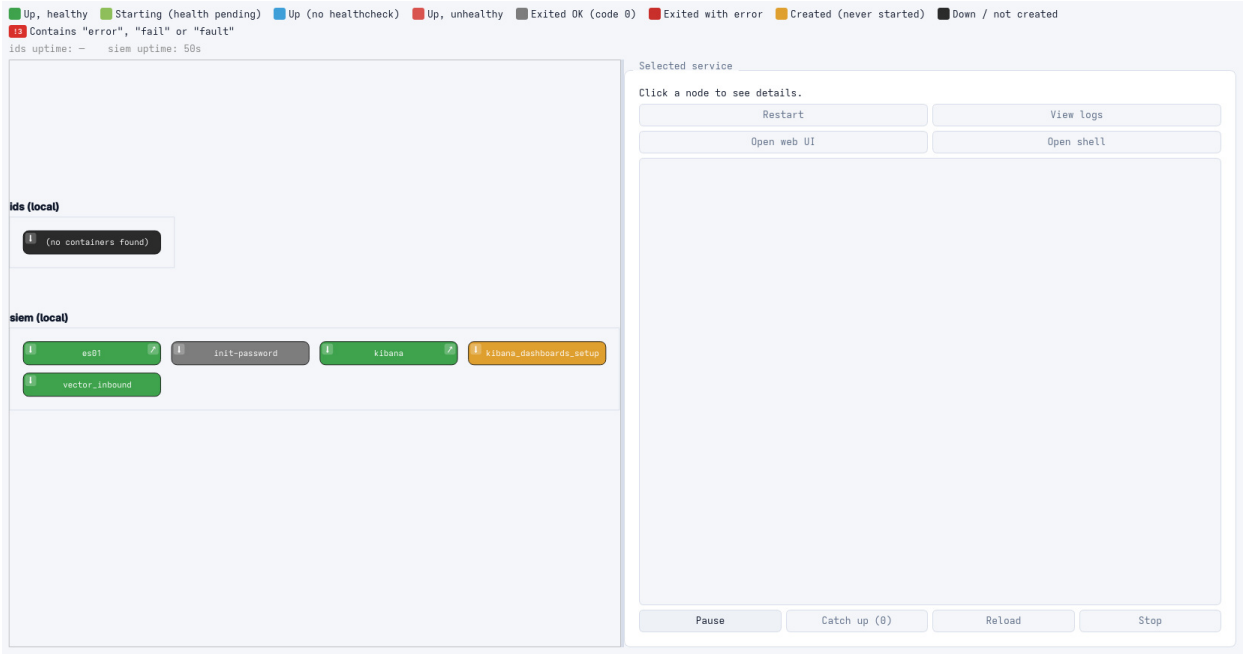
Before: stack idle / not yet changed



Starting: containers coming up



All containers healthy



4. Event log

Marker	Elapsed	Event
1	0s	init-password started
1	0s	kibana_dashboards_setup started
1	0s	es01 started
1	0s	kibana started
1	0s	vector_inbound started
2	20s	es01 healthy
3	22s	kibana running, health pending
3	22s	vector_inbound running, health pending
4	25s	vector_inbound healthy
5	49s	kibana healthy
5	49s	init-password stopped